

Diggaren

Om tjänsten

Syfte

Tjänsten "Diggaren" är utvecklad för att ge användare en smidig och användarvänlig plattform för att utforska och njuta av musik som spelas på svenska radiokanaler P2 och P3. Syftet är att förenkla processen för att hitta information om aktuella och föregående låtar och artister, samt att ge användarna en möjlighet att snabbt integrera dessa låtar med sina personliga spellistor på Spotify. Genom att koppla radio och digitala plattformar strävar tjänsten efter att förena traditionella musikmedier med moderna streamingtjänster. Tjänsten riktar sig till musikälskare som vill utforska musik från Sveriges Radio och integrera detta med sina personliga spellistor på Spotify.

Den primära målsättningen är att:

- Hjälpa användare att upptäcka nya låtar och artister som spelas på Sveriges Radio.
- Skapa en sömlös anslutning där användarna kan lyssna på låtarna i Spotify som de ser på radion.
- Utforma en plattform som är intuitiv, responsiv och lätt att navigera, oavsett vilken enhet som används.

Beskrivning av Applikationen

Startsida

Startsidan ger användarna en välkomnande introduktion till tjänsten. Här finns en tydlig och enkel knapp som tar användaren vidare till nästa sida, där applikationens huvudsakliga funktioner presenteras.

Diggarensida

Diggaren sidan innehåller en navigeringsbar och knappar för att enkelt välja mellan radiokanalerna P2 och P3. Dessutom finns det en sektion med information om hur användaren kan kontakta oss vid behov.

P2 och P3

På sidorna för P2 och P3 kan användaren se detaljerad information om den låt som spelas för tillfället, inklusive låttiteln och artistnamn, samt information om föregående låt.

Användaren kan välja att:

- **Lyssna på den aktuella låten på Spotify** genom att klicka på knappen "*Lyssna på Spotify*".
- **Lyssna på föregående låt på Spotify** genom att klicka på knappen "*Lyssna på tidigare låt på Spotify*".

Den tekniska lösningen

1. Vilka tekniker ni har använt för att implementera applikationen (frontend och backend)

Applikationen är byggd med hjälp av både moderna backend- och frontend-tekniker. På backend-sidan har vi använt Python och ramverket FastAPI för att hantera HTTP-förfrågningar och API-rutter. FastAPI är ett snabbt och flexibelt ramverk som underlättar skapandet av starka och skalbara applikationer. För att rendera dynamiska HTML-sidor använder vi Jinja2Templates, vilket möjliggör att data från backend skickas vidare till frontend. Requests-biblioteket används för att kommunicera med externa API:er som Sveriges Radio och Spotify. Vi använder även Uvicorn som ASGI-server för att köra applikationen och Spotipy som bibliotek som används för

att interagera med Spotify API. StaticFiles används för att hantera statiska filer som CSS och JavaScript. SessionMiddleware hanterar sessionsdata.

På frontend-sidan används HTML för att skapa sid-strukturen och Bootstrap 5 för att skapa en responsiv och visuellt tilltalande design. Applikationen har också en egen CSS-fil som innehåller anpassade stilar. Bootstrap-bibliotekets JavaScript används för att implementera interaktiva komponenter som navigationsmenyer.

2. En beskrivning av de datakällor/externa API:er ni använder

Applikationen använder två huvudsakliga externa API:er: Sveriges Radio API och Spotify API. Sveriges Radio API används för att hämta information om vilken låt som spelas just nu och vilken låt som spelades tidigare på deras kanaler P2 och P3. API:et tar kanal-ID:n (t.ex. 2562 för P2 och 164 för P3) som parameter och returnerar detaljer om låttitlar och artister i JSON-format.

Spotify API används för att fylla ut denna information genom att hitta låtar i Spotifys databas och generera lyssna länkar. Två olika API:er från Spotify används: ett för autentisering och åtkomsttoken och ett för att söka efter låtar. Vid en sökning skickas låttiteln och artistnamnet från Sveriges Radio API till Spotify som en kombinerad parameter (`track:{song_name} artist:{artist_name}`). Om Spotify hittar en matchande låt returnerar den en lyssningslänk som kan visas för användaren.

3. Om ni behandlar den data ni använder, ska även detta beskrivas.

Ja, data från både Sveriges Radio API och Spotify API behandlas innan den presenteras för användaren. För Sveriges Radio API bearbetas JSON-svaret för att hämta specifika fält, som låttitel och artist för både nuvarande och föregående låtar. Om datan från API:et saknas eller är tom, tillhandahålls fallback-värden, exempelvis "Ingen låt spelas just nu" för titeln eller "Okänd artist" för artistnamnet.

Autentisering mot Spotify hanteras på applikationsnivå (Client Credentials). Backend hämtar åtkomsttoken med CLIENT_ID/CLIENT_SECRET och använder titel/artist från Sveriges Radio som sökparametrar i formatet `track:{song_name} artist:{artist_name}` för att hitta en matchande låt i Spotify. Om en matchning hittas extraheras en lyssningslänk. Om ingen matchning finns,

informeras användaren om att låten inte är tillgänglig på Spotify. Denna bearbetade data skickas sedan till HTML-sidor via Jinja2-templates och visas dynamiskt i gränssnittet, vilket gör att användaren enkelt kan se låtinformation och, om tillgängligt, klicka på Spotify-länkar för att lyssna på musiken.

Användarmanual

Installationsinstruktioner

Steg 1. Extrahera den nedladdade zipfilen viktigt att den heter projektw.zip

Steg 2. Öppna den nedladdade mappen projektw i ditt programmeringsprogram.

Steg 3. Öppna en ny terminal i det program du programmerar i.

Steg 4. Efter det skriver du in (pip install -r requirements.txt) eller (pip3 install -r requirements.txt)

(Steg 5.) Det kan ibland behövas installera spotipy uvicorn fastapi. pip install spotipy uvicorn fastapi eller pip3 install spotipy uvicorn fastapi

Nu ska allt nödvändigt vara installerat för att kunna köra programmet.

!!! Vid körfel byt directory på nedan i main.py och pages.py till:

```
templates = Jinja2Templates(directory="../templates")  
app.mount("/static", StaticFiles(directory="../static"), name="static")
```

Alternativ körning är:

Steg 1. Extrahera den nedladdade zipfilen viktigt att den heter projektw.zip

Steg 2. Öppna den nedladdade mappen projektw i mappen projektw i ditt programmeringsprogram.

Steg 3. Öppna en ny terminal i det program du programmerar i och skriv in: cd /Users/YOURNAME/Downloads/projektw/python (Viktigt att det står ditt användarnamn för din dator samt projektw och inte projektw 1 som det kan stå om det laddats ner tidigare.)

Steg 4. Efter det skriver du in (pip install -r requirements.txt) eller (pip3 install -r requirements.txt)

(Steg 5.) Det kan ibland behövas installera spotipy uvicorn fastapi. pip install spotipy uvicorn fastapi eller pip3 install spotipy uvicorn fastapi

Nu ska allt nödvändigt vara installerat för att kunna köra programmet.

Steg 6. efter det skriver du in i terminalen: python main.py eller python3 main.py (Då kan ovan ändring med directory behövas)

Körinstruktioner

Kör programmet main.py och skriv in <http://127.0.0.1:8080/> i valfri webbläsare. Alternativt kör den via terminalen med ovan instruktioner (testa båda om ena inte fungerar)

Du möts av startsidan med enbart en knapp “Jag vill digga nu” som tar dig till nästa sida (tryck på den).

Nästa sida är sidan du kan välja ifall du vill kolla upp låtar på p2 eller p3. Tryck på en av dem.

Nu öppnas sidan upp för respektive p2 eller p3 där nuvarande låt som spelas på radion visas, samt föregående låt. Testa att trycka på “lyssna på Spotify”, denna knapp ska visa upp samma låt på Spotify som visades på p2 eller p3.

Det finns även möjlighet att hitta våra kontaktuppgifter på “kontakta oss” samt gå tillbaka till startsidan på “Diggaren”.

Testa även skriv in <http://127.0.0.1:8080/api/channels/p2> (eller p3) för att se vårt API i JSON.

Nu har du kört igenom hela vårt program!

API-dokumentation

Översikt

API:t är byggt med FastAPI och exponerar data om aktuell och föregående låt för P2 och P3. Det returnerar JSON och används även av webbgränssnittet som ett mellanlager, så att sidorna inte anropar externa API:er direkt.

Bas-URL lokalt: <http://127.0.0.1:8080/> (körs via uvicorn).

Tillgängliga kanaler: p2 (2562), p3 (164).

API-struktur och endpoints:

När vi designade API:t för Diggaren utgick vi från principerna om tydlighet, enkelhet och återanvändbarhet. Målet var att skapa ett API som både kunde underhålla vår egen webbapplikation med data och vara användbart för eventuella externa klienter.

1. REST-inspirerad struktur

Vi har valt att följa en REST-liknande design där varje resurs (radiokanal och dess låt-data) har en tydlig URL-struktur:

- /api/channels – Lista över tillgängliga kanaler.
- /api/channels/{channel} – Komplet låtinfo (aktuell och föregående) för en specifik kanal.
- /api/channels/{channel}/current – Endast aktuell låt för kanalen.
- /api/channels/{channel}/previous – Endast föregående låt för kanalen.

Denna struktur gör det enkelt att förstå vad varje endpoint gör och det möjliggör både breda och smala anrop beroende på klientens behov.

2. Separering av ansvar

API:t är byggt som ett mellanlager mellan externa tjänster (Sveriges Radio och Spotify) och användargränssnittet.

- All kommunikation med externa API:er sker i backend, vilket innebär att frontend aldrig hanterar API-nycklar eller autentisering direkt.
- Detta ger bättre säkerhet, centraliserad felhantering och en tydlig plats för databehandling innan den presenteras för användaren.

3. Enhetligt svar format

Alla svar från API:t returneras i JSON-format med konsekvent struktur.

- Vid lyckade anrop innehåller svaret tydliga fältnamn (title, artist, spotify_link).

- Vid fel returneras ett JSON-objekt med error och code, vilket gör det enkelt för klienter att hantera felen.
- HTTP-statuskoder används konsekvent (200 för OK, 404 för ogiltig kanal, 500 för serverfel).

4. Databearbetning och fallback-värden

API:t hämtar rådata från Sveriges Radio och förädlar den genom att:

- Extrahera relevanta fält (titel och artist).
- Lägga till en Spotify-länk om en träff hittas via Spotify API.
- Använda fallback-texter som "Ingen låt spelas just nu" och "Okänd artist" när data saknas.

Detta säkerställer att API:t alltid returnerar användbar och konsekvent information.

5. Asynkron design för prestanda

Anrop till Sveriges Radio och Spotify görs asynkront (via `httpx.AsyncClient`), vilket gör att API:t kan svara snabbt även när externa tjänster har viss svarstid. Detta minskar risken för flaskhalsar och förbättrar användarupplevelsen.

6. Utbyggbarhet

Strukturen med separata moduler (`radio.py`, `spotify.py`, `mashup.py`) gör det enkelt att:

- Lägga till fler radiokanaler.
- Koppla på fler musiktjänster än Spotify.
- Bygga ut API:t med fler endpoints utan att påverka befintlig funktionalitet.

Swagger: <https://app.swaggerhub.com/apis/private-29c/benjaminorobinsdiggaren/1.0.0#/info>

Felhantering och svar:

Felhanteringen i API:et är centraliserad och hanterar både undantag som kastas av FastAPI och de som kommer från Starlette, vilket innebär att även 404-fel från routingen fångas upp och bearbetas. Alla fel svarar i ett enhetligt JSON-format med fält som `error`, `code` och, vid behov, `path` eller `details`. Ett särskilt fall gäller för anrop till `/api/channels/{channel}/{action}` där `action`

är felstavad eller inte tillhör de tillåtna värdena (current eller previous). I dessa fall returnerar API:et statuskod 400 i stället för 404, eftersom resursmönstret finns men parametern är ogiltig. Felmeddelandet är tydligt och kan även innehålla ett automatiskt framtaget förslag på rätt stavning. Vid valideringsfel, som när en parameter har fel datatyp, returneras statuskod 422 med detaljerad information om felet. Om API:et inte kan hämta data från radiotjänsten eller om ett oväntat fel inträffar, returneras statuskod 500 med ett generiskt felmeddelande för att undvika att exponera intern information. Även andra HTTP-fel, som 401 eller 403, hanteras och returneras med samma konsekventa struktur. När allt fungerar som det ska returnerar API:et en JSON-respons med information om den nuvarande och föregående låten, inklusive titel, artist och, om tillgängligt, en länk till låten på Spotify.

Respons Format:

Svaren från API:et är i JSON-format. Denna struktur är enkel att parse och hantera för både frontend-applikationer och externa klienter.